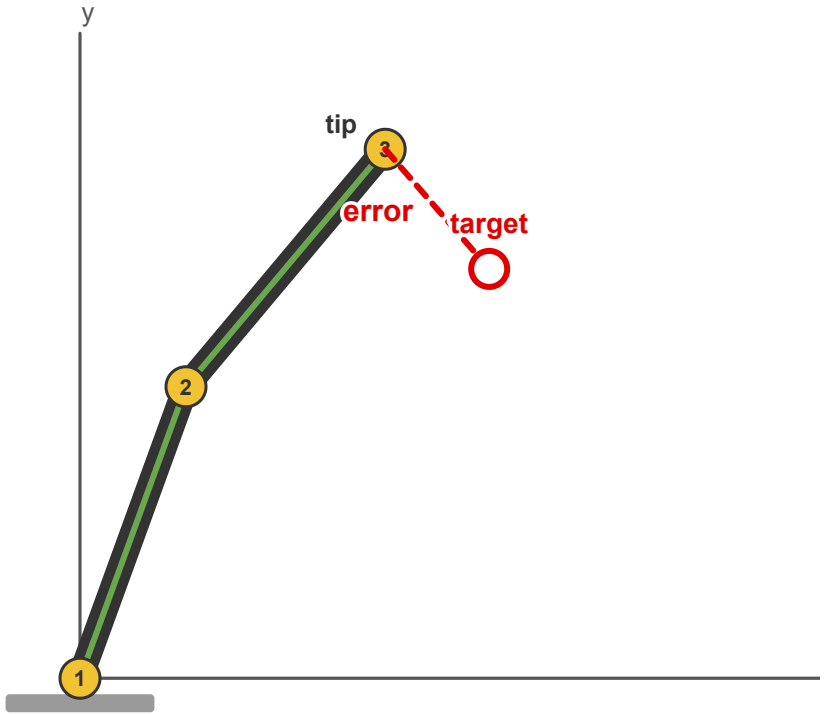
ROBOTIC ARM COURSE · LESSON 7 OF 12

The Gradient Descent Approach to Inverse Kinematics

Iterative and numerical IK methods: guess, measure the error, improve

Day 1 · Gradient descent on the flat arm

Day 2 · The 3D arm in code



RECAP · LESSON 6

Where Lesson 6 Left Off

GIVEN · THE TARGET

(x, y, z) = (301.78, 301.78, 176.78)

→

TRIANGLES GAVE

B = S = E = 45°

Trigonometry gave an exact answer, but only for **this** arm. Every new joint needs new triangles and new special cases.

Today's method needs just one thing we already have: **forward kinematics** from Lessons 4 and 5. Instead of solving for the angles, we **improve a guess** until the tip lands on the target.

DAY 1 · THE IDEA

Guess, Check, Improve

Think of the game "hot and cold". Each round tells you whether you are getting closer.

1 · GUESS
joint angles

→

2 · FK
where is the tip?

→

3 · ERROR
how far from the target?

→

4 · NUDGE
angles that shrink the error

Then go back to step 1 and repeat, until the error is tiny.

The clever part is step 4: **which way** should each joint turn, and **how far**? Gradient descent answers both.

DAY 1 · THE ERROR

Measuring the Error

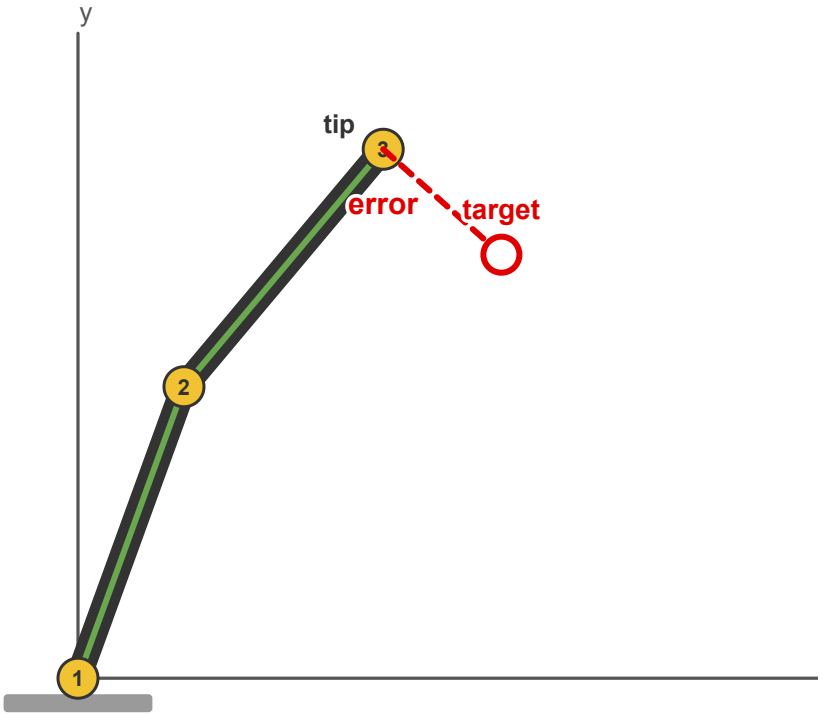
The **error** is the distance from the tip to the target. We score a guess with the **cost**, the squared distance in link lengths:

$$C = (X - X_t)^2 + (Y - Y_t)^2$$

with each distance divided by L = 250 mm

Why squared? It is never negative, it is 0 only when the tip is on the target, and it makes a smooth bowl to roll down.

Goal: find angles that make C as small as possible.



DAY 1 · DOWNHILL

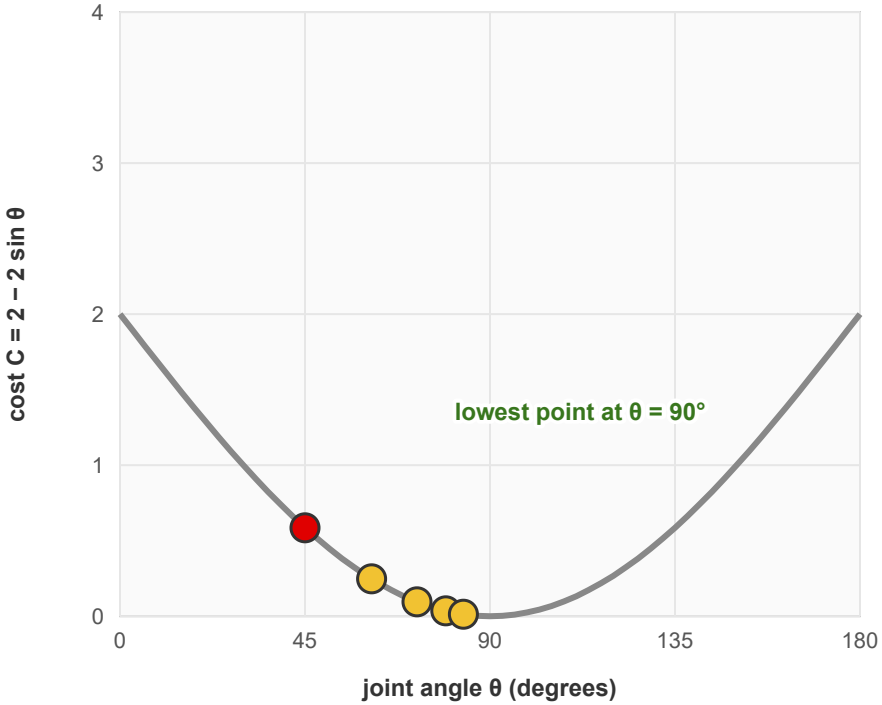
Rolling Downhill

Plot the cost against a joint angle. It looks like a **valley**. The best angle is the bottom.

1. Measure the **slope** where you stand.
2. Step **the opposite way** (downhill).

$$\theta_{new} = \theta - \eta \times \text{slope}$$

η (eta) is the **learning rate**: the size of each step. The slope is the gradient.



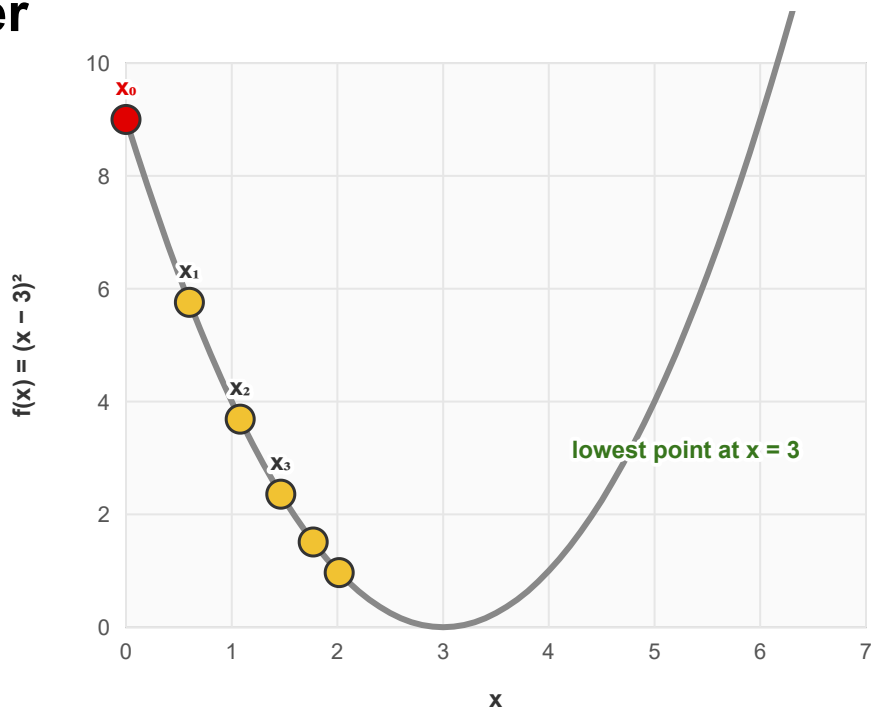
DAY 1 · WORKED EXAMPLE

Gradient Descent on One Number

Find the lowest point of **f(x) = (x − 3)²**. The slope is **2(x − 3)**. Start at x = 0 with η = 0.1.

STEP	X	SLOPE	NEW X
1	0	-6	0.6
2	0.6	-4.8	1.08
3	1.08	-3.84	1.464
4	1.464	-3.072	1.7712
5	1.7712	-2.458	2.017

x creeps toward 3. Steps shrink as the slope flattens near the bottom.



DAY 1 · TRY IT YOURSELF

Try It Yourself #1

Same function, $f(x) = (x - 3)^2$, slope $2(x - 3)$, starting at $x = 0$.

(a) $\eta = 0.25$ **(b) $\eta = 1$**

SOLUTION

(a) $x_1 = 0 - 0.25 \times (-6) =$ **1.5**; $x_2 = 1.5 - 0.25 \times (-3) =$ **2.25**

(b) $x_1 = 0 - 1 \times (-6) =$ **6**; $x_2 = 6 - 1 \times 6 =$ **0**; then 6, 0, 6, 0…

$\eta = 1$ bounces forever

A step that is too big jumps over the bottom, so it never settles.

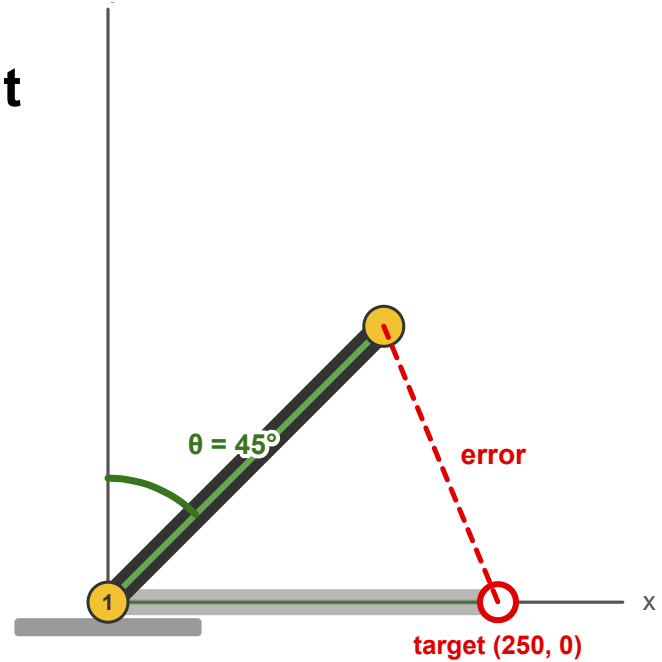
DAY 1 · A ONE-JOINT ARM

Gradient Descent on a Single Joint

A single 250 mm link turns by θ from vertical. The target is at (250, 0). Here the cost is **C = 2 − 2 sin θ** and its slope is **−2 cos θ**. Start at θ = 45°, η = 0.2.

STEP	Θ	SLOPE	NEW Θ (RAD)	NEW Θ
1	45.00°	-1.4142	1.0682	61.21°
2	61.21°	-0.9633	1.2609	72.24°
3	72.24°	-0.6099	1.3829	79.23°
4	79.23°	-0.3736	1.4576	83.51°
5	83.51°	-0.2259	1.5028	86.10°

The update uses radians (180° = π rad ≈ 3.1416). Convert back at the end. θ heads to 90°.



DAY 1 · MEASURING THE SLOPE

Measuring the Slope by Nudging

We do not need calculus. **Nudge** the joint a tiny amount h and watch the cost change:

$$\text{slope} \approx \frac{C(\theta + h) - C(\theta)}{h}$$

At θ = 45° (0.7854 rad), with h = 0.01:

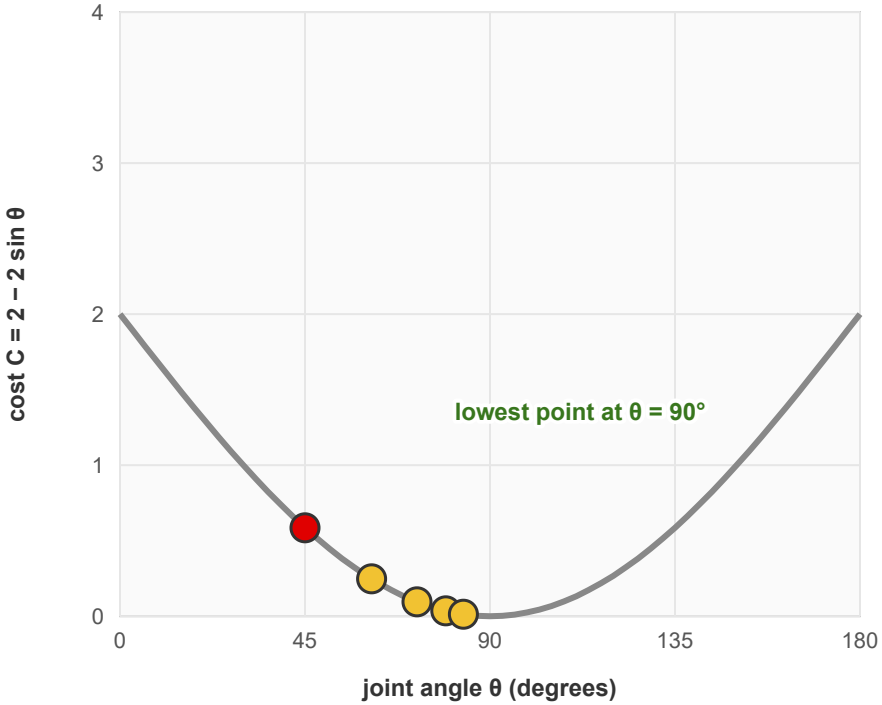
C(θ) = 0.58579

C(θ + h) = 0.57172

slope ≈ (0.57172 − 0.58579) ÷ 0.01 = **-1.407**

The exact slope −2 cos 45° = −1.414.

This only needs the cost, which needs only FK. So it works for **any** arm.



DAY 1 · TRY IT YOURSELF

Try It Yourself #2

One joint, target (250, 0), cost $C = 2 - 2 \sin \theta$, slope $-2 \cos \theta$, $\eta = 0.2$. Start at $\theta = 60°$.

Find θ after (a) one step and (b) two steps.

SOLUTION

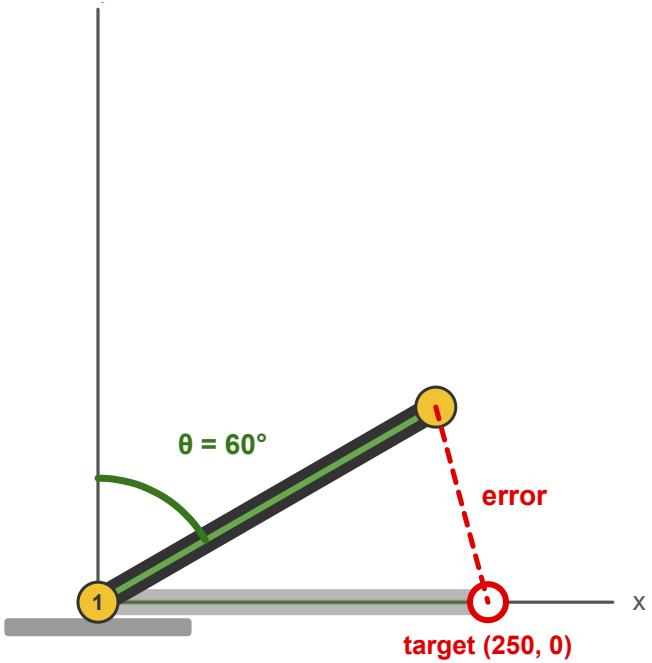
(a) slope = $-2 \cos 60° = -1$, so $\theta = 1.0472 + 0.2 \times 1 = 1.2472$ rad

$\theta = 71.46°$

(b) slope = $-2 \cos 71.46° = -0.636$, so $\theta = 1.2472 + 0.2 \times 0.636 = 1.3744$ rad

$\theta = 78.75°$

Each step moves closer to 90°.



DAY 1 · THE FLAT ARM

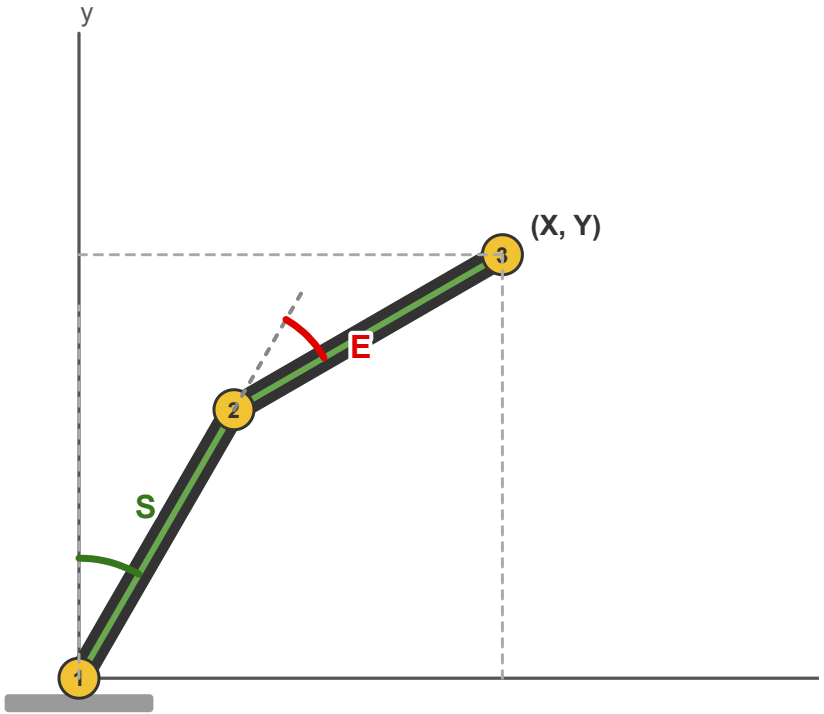
The Flat Arm: Two Joints

For the flat arm of Lesson 4, the tip is (use S from vertical and E, as in Lesson 6):

$$X = L \sin S + L \sin(S + E)$$
$$Y = L \cos S + L \cos(S + E)$$

Check S = E = 45°: X = 176.78 + 250 = 426.78 and Y = 176.78 + 0 = 176.78. ✓

Now the cost depends on **two** angles. The **gradient** is a pair of slopes, one per joint: how the cost changes when you nudge S, and when you nudge E.



DAY 1 · WORKED EXAMPLE · SETUP

Worked Example: Setup

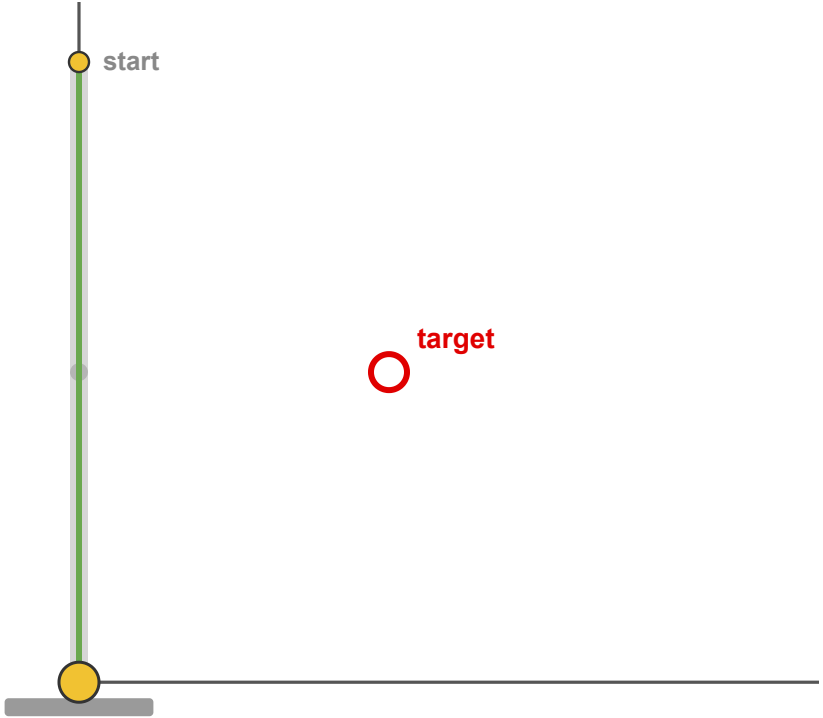
Reach the target **(X, Y) = (250, 250)** with L = 250 mm.

GIVEN

Start: **S = 0, E = 0** (arm straight up, tip at (0, 500))

Learning rate: **η = 0.1**

Cost at the start: the tip is off by (0 − 250, 500 − 250) = (−250, 250), which is (−1, 1) link lengths. So **C = 1 + 1 = 2**.



DAY 1 · WORKED EXAMPLE · STEP 1

Worked Example: Measure the Slopes

Nudge each joint by h = 0.01 rad and see what the cost does. Start: C = 2.

NUDGE S

C after nudging S = **1.9602**

$\text{slope}_S = (1.9602 - 2) \div 0.01 \approx \mathbf{-4}$

NUDGE E

C after nudging E = **1.9800**

$\text{slope}_E = (1.9800 - 2) \div 0.01 \approx \mathbf{-2}$

Both slopes are negative, so raising S and E lowers the cost. S matters twice as much as E. The gradient is **(−4, −2)**.

DAY 1 · WORKED EXAMPLE · STEP 2

Worked Example: Take a Step

Update both joints at once with η = 0.1:

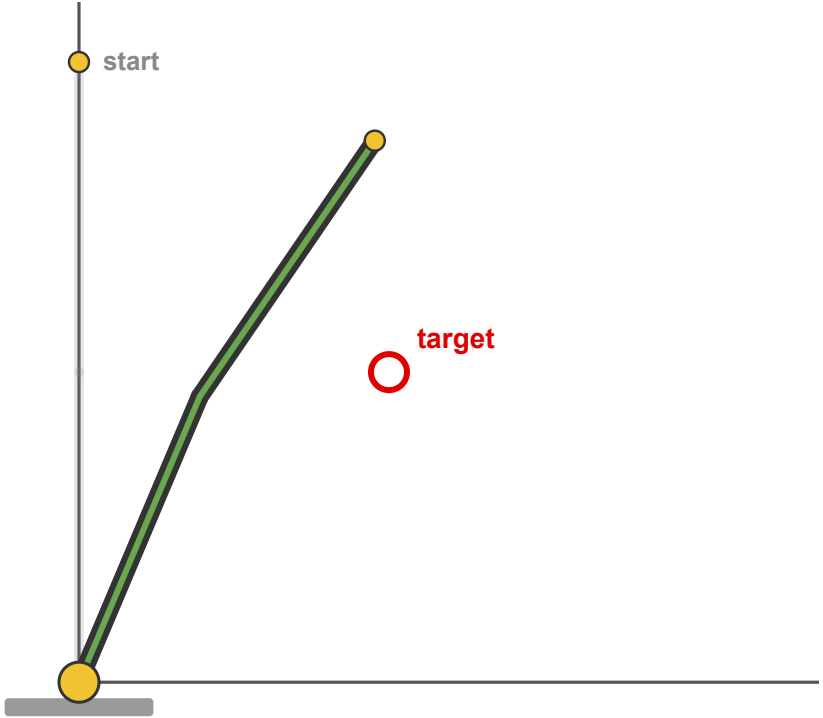
THE UPDATE

$S_{new} = 0 - 0.1 \times (-4) =$ **0.4 rad = 22.92°**

$E_{new} = 0 - 0.1 \times (-2) =$ **0.2 rad = 11.46°**

The tip moves from (0, 500) to **(238.5, 436.6)**. The error drops from 353.6 mm to **187.0 mm**.

Then repeat: measure new slopes, take another step.

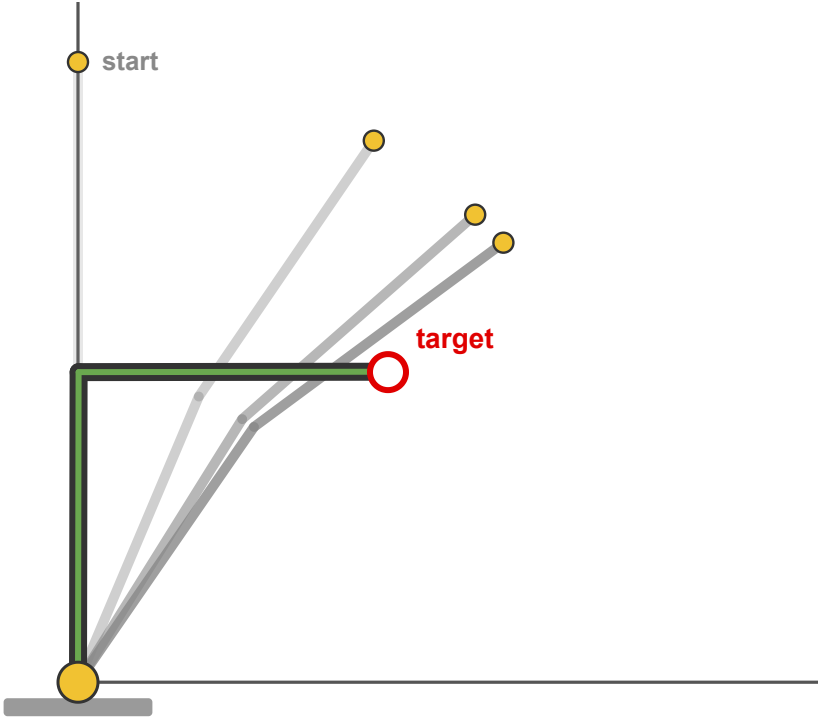


DAY 1 · WORKED EXAMPLE · REPEAT

Worked Example: Keep Going

STEP	S	E	TIP (X, Y)	ERROR
0	0.00°	0.00°	(0.0, 500.0)	353.6
1	22.92°	11.46°	(238.5, 436.6)	187.0
2	32.00°	16.72°	(320.3, 376.9)	145.1
3	34.59°	18.97°	(343.1, 354.3)	139.8
10	32.71°	26.51°	(349.9, 338.3)	133.3
30	19.18°	54.67°	(322.3, 305.6)	91.2
106	0.09°	89.85°	(250.4, 250.3)	0.5

Big steps at first, smaller ones later. It lands after **106** steps within 0.5 mm. The answer, S ≈ 0°, E ≈ 90°, matches Lesson 6.



DAY 1 · CHECK

Same Answer, No Triangles

LESSON 6 · TRIGONOMETRY

S = 0°, E = 90°

Law of cosines, atan2 and a few formulas.

Exact in one pass. Needs the geometry of this arm.

LESSON 7 · GRADIENT DESCENT

S ≈ 0.1°, E ≈ 89.9°

Guess, nudge, step, repeat.

Approximate. Needs only FK and a loop.

Same target, same final pose. Gradient descent trades **exactness** for **generality**: if you can compute FK for an arm, you can do IK for it.

DAY 1 · TRY IT YOURSELF

Try It Yourself #3

Target (341.51, 341.51), start S = E = 0, η = 0.1. The slopes are **$\text{slope}_S = -5.46$** and **$\text{slope}_E = -2.73$**.

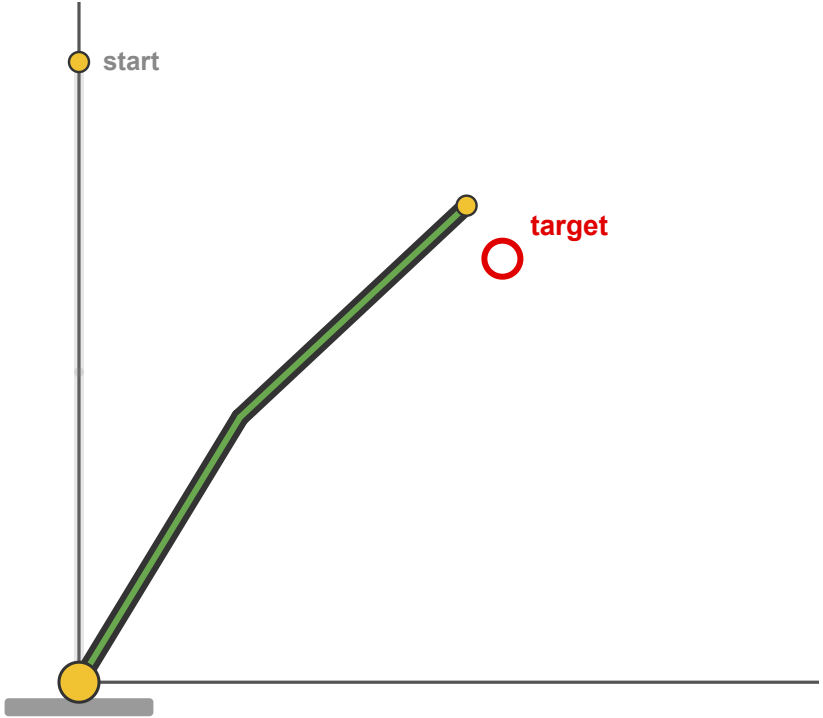
Find the new S and E in radians and degrees. (180° = π rad.)

SOLUTION

S = 0 − 0.1 × (−5.46) = **0.546 rad** = 31.3°
E = 0 − 0.1 × (−2.73) = **0.273 rad** = 15.6°

S ≈ 31.3°, E ≈ 15.6°

The tip moves to (312.6, 384.2), and the error falls from 376.5 mm to 51.6 mm.



DAY 1 · RECAP

Day 1 Recap

COST

C = squared distance to the target

A bowl whose bottom is the answer.

SLOPE

Nudge a joint, watch the cost

slope ≈ (C(θ + h) − C(θ)) ÷ h. Only FK is needed.

STEP

$\theta_{new} = \theta - \eta \times \text{slope}$

Go downhill. Do every joint at once.

REPEAT

Until the error is tiny

η too big bounces. Too small crawls.

Next time: the real arm's base turns too. We let a computer do the nudging for all three joints.

DAY 2 · EXTENDING TO 3D

Extending to 3D

Now the arm has **three** joints: B, S and E. Nothing in the method changes:

1. **FK** from Lesson 4 turns (B, S, E) into the tip (x, y, z).
2. **Cost** C = squared distance from the tip to the target (x, y, z), in link lengths.
3. **Gradient** = three slopes, found by nudging B, S and E in turn.
4. **Step** all three angles downhill, then repeat.

By hand that is a lot of arithmetic, so we hand the loop to Python.

DAY 2 · THE CODE

Let the Computer Nudge

```
def cost(angles, target):
    e = (tip(angles) - target) / L
    return e @ e        # squared distance

def gradient(angles, target, h=1e-4):
    g = np.zeros(3)
    for i in range(3):           # B, S, E
        nudge = np.zeros(3)
        nudge[i] = h
        g[i] = (cost(angles + nudge, target)
              - cost(angles - nudge, target)) / (2*h)
    return g

# the loop inside ik_gd
for step in range(max_steps):
    if error < tol:
        break
    angles = angles - eta * gradient(angles, target)
```

WHAT EACH PART DOES

tip is Lesson 4's FK (three DH matrices).

cost scores a guess.

gradient nudges each joint up and down by h, and divides the change in cost by 2h. That is the slope.

The loop is the whole method: stop when the error is under the tolerance, otherwise step downhill.

The helper file gd_ik.py (in the Appendix of the lesson plan) holds all of this.

DAY 2 · CHECK WITH CODE

Check It with Code

```
from gd_ik import ik_gd, tip
import numpy as np

angles, steps, error = ik_gd((301.78, 301.78, 176.78))
print([round(a, 2) for a in angles])
print(steps, round(error, 2))

# check with FK (Lesson 4)
print(tip(np.radians(angles)).round(2))
```

OUTPUT

```
[45.0, 45.15, 44.7]
180 0.49
[302.11, 302.11, 176.93]
```

Angles within 0.2° of Lesson 6's (45°, 45°, 45°) after 180 steps. FK puts the tip within 0.5 mm of the target.

DEFAULTS

start = (10, 10, 10) degrees, η = 0.1, tolerance = 0.5 mm, at most 1000 steps.

DAY 2 · WATCHING IT WORK

Watching the Error Fall

Each step shrinks the error. From the worked example, with η = 0.1:

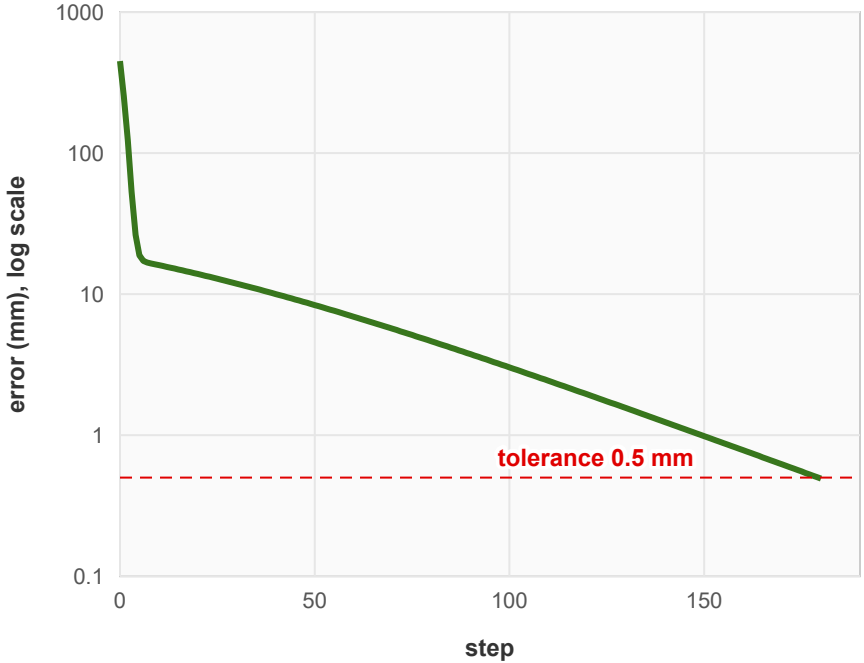
Start: **449 mm** off

Step 10: **16 mm**

Step 50: **8.4 mm**

Step 180: **under 0.5 mm**

Fast at first, then a slow crawl: the valley gets flat near the bottom, so the slope (and the step) gets tiny.



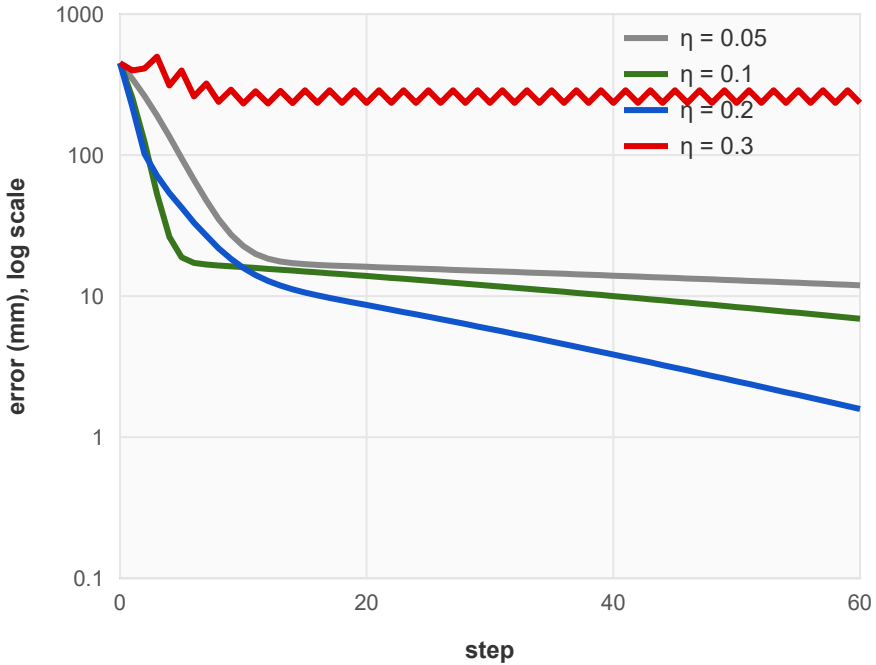
DAY 2 · TUNING

Choosing the Learning Rate η

Same target, same start, five learning rates:

LEARNING RATE	STEPS TO REACH 0.5 MM
η = 0.02	906 steps
η = 0.05	362 steps
η = 0.1	180 steps
η = 0.2	85 steps
η = 0.3	never settles

Too small: safe but slow. **Too big**: it overshoots and bounces, like η = 1 in Try It Yourself #1. We use **η = 0.1**.



DAY 2 · STOPPING

When Do We Stop?

The loop needs a rule, or it could run forever.

1. **Tolerance**: stop when the error is small enough, like 0.5 mm.
2. **Step limit**: stop after a set number of steps, like 1000, even if the error is still large.

Always look at the final error. Stopping at the step limit does not mean the answer is right. `ik_gd` returns the error so you can check.

A real robot also has a time budget, so it needs the step limit.

DAY 2 · TRY IT YOURSELF

Try It Yourself #4

Run `ik_gd((187.5, 324.76, 216.51))` with the default settings. Compare with Lesson 6's Try It Yourself #4.

SOLUTION

Output: [60.0, 30.12, 59.77] in 148 steps, error 0.49 mm

B ≈ 60°, S ≈ 30°, E ≈ 60°

The same as the trigonometry answer (60°, 30°, 60°), to within the tolerance.

Try η = 0.2. It finishes in about half the steps.

DAY 2 · THE START MATTERS

Different Start, Different Answer

Target **(−324.76, 187.50, 216.51)**, Lesson 6's answer: B = 150°, S = 30°, E = 60°.

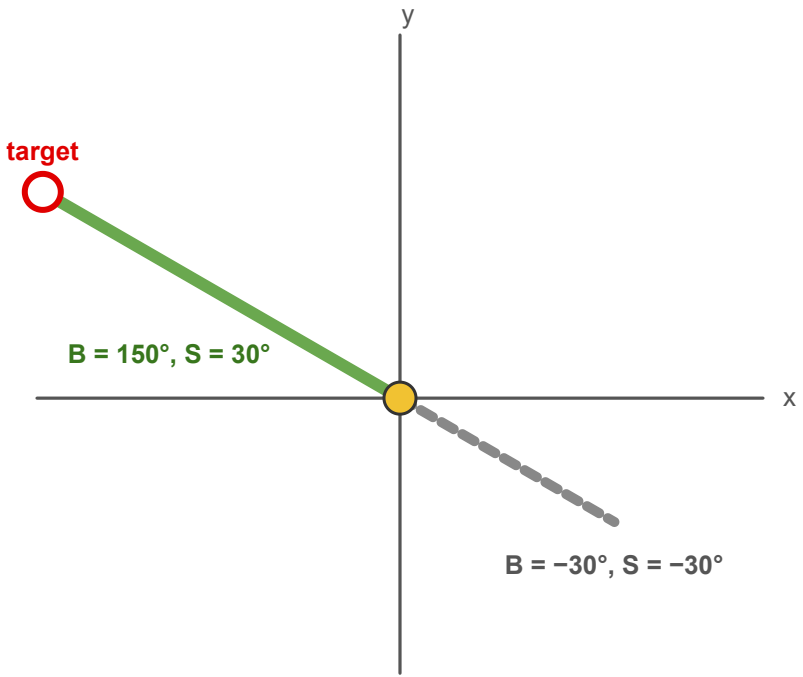
Start (10, 10, 10) → B = −30°, S = −30°, E = −59.8°

The base turns the "short way" and the arm leans back.

Start (100, 10, 10) → B = 150°, S = 30.1°, E = 59.8°

Starting near the target side gives the Lesson 6 answer.

Both put the tip on the target. Gradient descent goes to the nearest valley, so choose a start near the pose you want, and check joint limits.



DAY 2 · TRY IT YOURSELF

Try It Yourself #5

Run `ik_gd((-125, 216.51, 250), start=(100, 10, 10))`. Compare with Lesson 4's Try It Yourself #5 (B = 120°, S = 0°, E = 90°).

SOLUTION

Output: [120.0, 0.1, 89.84] in 100 steps, error 0.50 mm

B ≈ 120°, S ≈ 0°, E ≈ 90°

The base ends past the y axis. Now try start=(10, 10, 10) and see which way B turns.

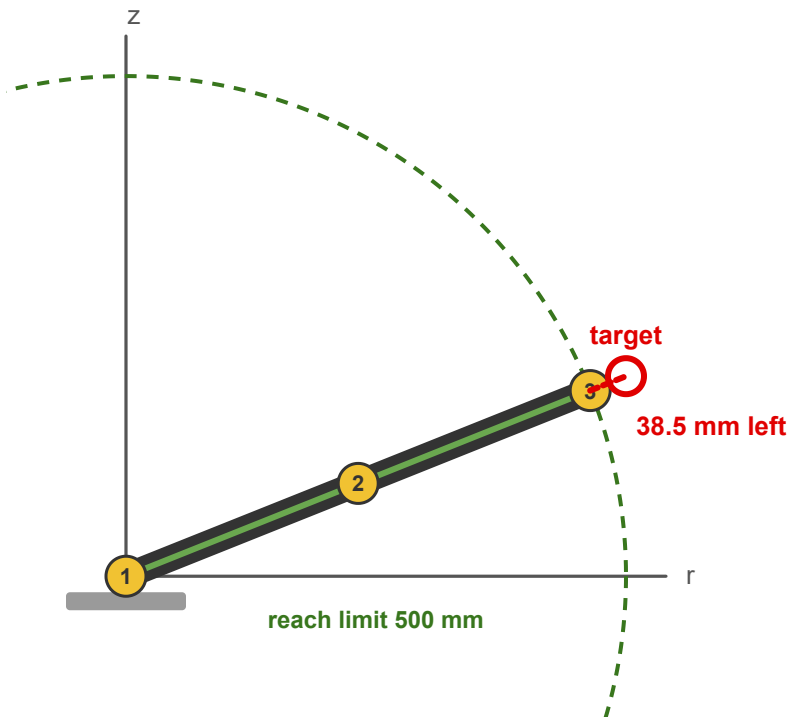
DAY 2 · OUT OF REACH

Out of Reach: No Error, Just a Gap

Try Lesson 6's unreachable target **(400, 300, 200)**. Trigonometry raised an error. Gradient descent does not:

> Output: [36.87, 68.2, 0.0], error **38.5 mm**
>
> It used all the steps and stopped at the closest point: arm fully straight (E = 0°) and pointing at the target.

Always check the final error. If it is bigger than the tolerance, the target is not reachable (or the loop got stuck).



DAY 2 · LIMITS

When Gradient Descent Struggles

SITUATION	WHAT HAPPENS
Target out of reach	ends with a leftover error
Target near full reach	very slow: 2000+ steps, error 0.75 mm
Learning rate too big	bounces, never settles
Learning rate too small	needs hundreds of steps
Different start	a different (valid) answer

Near full reach the arm is almost straight, where bending the elbow barely moves the tip. The slope is tiny, so so are the steps.

DAY 2 · CHECK

Always Check with FK

TARGET	GRADIENT DESCENT	FK (LESSON 4)
(301.78, 301.78, 176.78)	→ 45.0°, 45.2°, 44.7°	→ (302.11, 302.11, 176.93)

The loop already uses FK, so the check is built in. The final error is the distance between the target and FK of the answer: **0.49 mm**.

For a robot: if the error is under your tolerance, send the angles to the motors. If not, do not move.

DAY 2 · COMPARISON

Trigonometry vs. Gradient Descent

LESSON 6 · TRIGONOMETRY

+ Exact answer in one pass
+ Very fast
− New triangles for every new joint
− Special cases to handle by hand

LESSON 7 · GRADIENT DESCENT

+ Same loop for any arm: just supply FK
+ Unreachable targets fail gently
− Approximate and slower
− Needs a good η and a good start

If the arm had six joints, which method would you use? Why?

Gradient descent: the code barely changes. Only the FK function grows.

DAY 2 · BEYOND

Making It Faster

Plain gradient descent is the simplest numerical IK method. Real robots use the same loop with smarter steps:

1. **Warm start**: begin from the arm's current pose. The next target is usually close, so only a few steps are needed.
2. **Jacobian methods**: use the exact slopes of the tip, found with calculus, instead of nudging.
3. **Adaptive steps**: take big steps where the slope is steep and small ones where it is flat, to avoid η problems.

All of them are still: guess, measure the error, improve.

DAY 2 · WRAP-UP

What We Learned

1. Iterative IK **guesses, measures the error, and improves**, using only FK.
2. **Cost** = squared distance to the target. Its **slope** comes from nudging each joint.
3. $\theta_{new} = \theta - \eta \times slope$. Too big a η bounces, too small crawls.
4. Always **check the final error**, and mind the start pose.

NEXT TIME · LESSON 8

Putting IK to Work

We use our IK solvers to send real targets to the arm.